

Algos gloutons

27 juin 2022 16:52

Techniques de design d'algo

- Algos gloutons (1 semaine)
 - Diviser-pour-régner (2 sem.)
 - Prog. dynamique (2 sem.)
 - Algos probabilistes (1 sem?)
 - Force brute
-

Algos gloutons: algo qui construit une solution un elt à la fois.

À chaque étape, on choisit l'elt le + prometteur en ce moment.

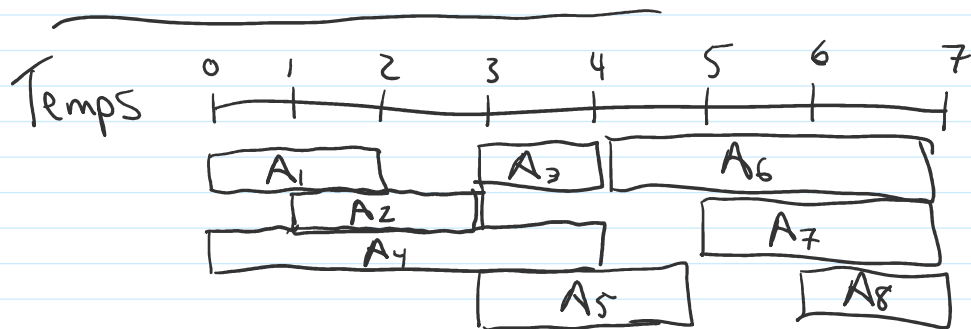
ex: Prim (ACM)

k-centres

Souvent, ça échoue. (contre-exemple)

Parfois, ça marche.

Pb de la sélection d'activité



But: choisir # max de A_i non-chevauchants

ex: $\{A_1, A_3, A_6\}$ $\{A_2, A_5, A_8\}$

Entrée: ensemble d'intervalles $A = \{(s_1, t_1), (s_2, t_2), \dots, (s_n, t_n)\}$
 $(s_i, t_i) = \{r \in \mathbb{R} : s_i < r < t_i\}$

Sortie: $S \subseteq A$ de taille max tel que

$\forall (s_i, t_i), (s_j, t_j) \in S$ avec $i \neq j$, on a
 $(s_i, t_i) \cap (s_j, t_j) = \emptyset$

glouton Long Min (A)

$S = []$

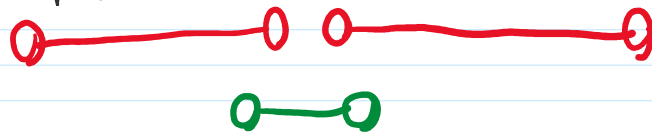
tant que $\exists (s_i, t_i) \in A$ tq. $S \cup \{(s_i, t_i)\}$ est valide

1 2 3 4 5 6 7

tant que $\exists (s_i, t_i) \in A$ t.q. $s \cup \{(s_i, t_i)\}$ est valide

| Soit (s_i, t_i) de longueur min t.q.
| $S \cup \{(s_i, t_i)\}$ est valide
| $S.append((s_i, t_i))$
+
return S

Ne fonctionne pas:



glouton retourne 1 act (vert)
optimal = 2 act (orange)

glouton 2: toujours choisir A_i avec temps de départ s_i minimum

Ne fonctionne pas:



$S = \{1, 2, 3, 5\}$



glouton 3: toujours choisir A_i avec le temps de fin minimum

actif Glouton (A)

trier A selon les t_i

$S = \text{set}()$

Trier A selon les L_i

$S = \text{set}()$

pour $i = 1..n$

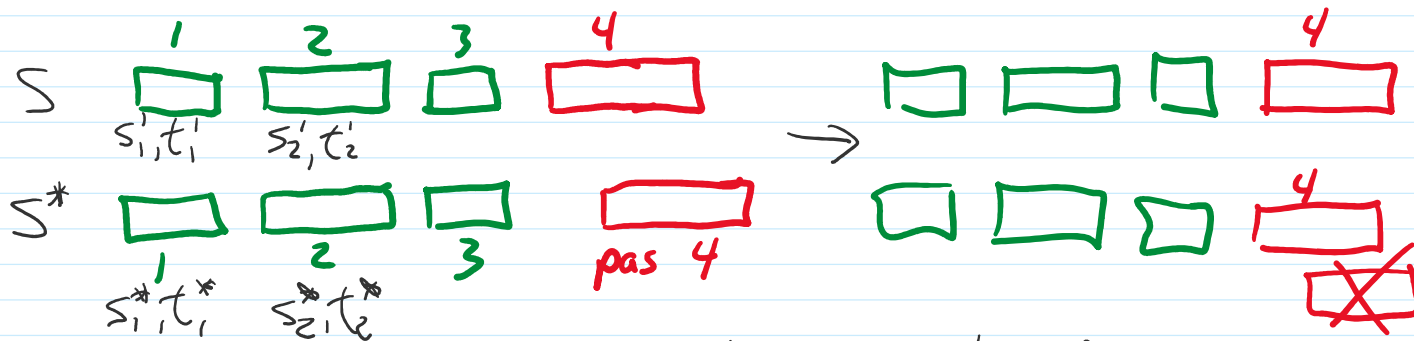
si $A[i]$ ^{n'intersecte} ne chevauche pas avec les elts de S

$S.add(A[i])$

return S

Comment montrer que cet algo fonctionne?

- On compare le S retourné par l'algo à une solution optimale S^* . But: montrer $|S| = |S^*|$.
↳ on ne connaît pas S^* (mais elle existe).
- On compare les elts de S dans l'ordre, /par/, et on s'arrête sur 1^{re} différence avec S^* .



Argumenter qu'on pourrait ajouter cette 1^{re} activité à S^* .

Argumenter qu'on pourrait ajouter cette 1^{re} activité à S^* .

Thm: actif Greedy retourne une solution optimale.

Preuve: soit $S \in A$ la sol. retournée par l'algo
soit $S^* \in A$ une sol. optimale.

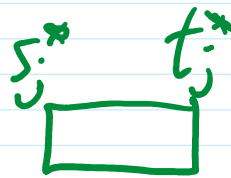
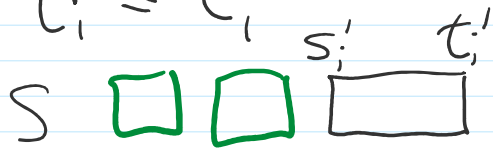
Soit $(s_1, t_1), (s_2, t_2), \dots, (s_k, t_k)$ les act. de S
dans l'ordre d'ajout.

Soit $(s_1^*, t_1^*), (s_2^*, t_2^*), \dots, (s_\ell^*, t_\ell^*)$ les act. de S^* ,
en ordre de t_i^* .

Soit i l'indice minimum tel que $(s_i, t_i) \neq (s_i^*, t_i^*)$

Si i n'existe pas, $S = S^*$ et on a fini (exercice).

Par le choix greedy de notre algo, on a
 $t_i \leq t_i^*$



- n (s_1, t_1) < (s_2, t_2) < ... < (s_i, t_i) < ... < (s_\ell, t_\ell) - ^

Considérons $(S^* \setminus \{(s_i^*, t_i^*)\}) \cup \{(s_i', t_i')\} = \hat{S}$

On doit argumenter que $\hat{S}$ est valide.

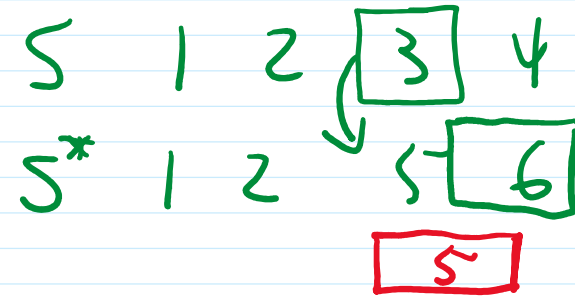
On voit que $\forall j > i$, on a $s_j^* \geq t_i^*$.

$\Rightarrow t_i' \leq t_i^* \leq s_j^* \Rightarrow (s_i', t_i')$ ne chevauche pas (s_j^*, t_j^*) .

$\Rightarrow (s_i', t_i')$ ne chevauche pas les (s_j^*, t_j^*) avec $j < i$

$\Rightarrow \hat{S}$ est valide.

Puisque $|\hat{S}| = |S^*|$ (exercice),
 $\hat{S}$ est solution alternative optimale.



De plus, $\hat{S}$ a un elt en commun
 avec notre S de plus. que S^* .

En répétant cet argument autant de fois que
 nécessaire, on arrive à un $\hat{S}$ dont les é/ls

nécessaire, on arrive à un S dont les e/s en commun avec notre S sont en fait tous les e/s de S . C'est-à-dire, $\hat{S} = S \Rightarrow S$ est optimal. $\square$

Preuve que l'algo de Prim fonctionne.

Rappel $\text{Prim}(G = (V, E, f))$

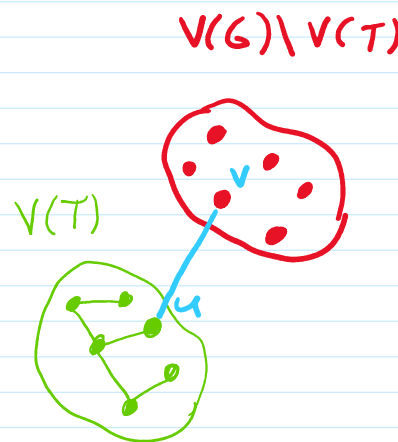
$$T = (\{v\}, \emptyset)$$

tant que $|E(T)| < n - 1$

trouver $uv \in E(G)$ t.q.

- $u \in V(T)$
- $v \in V(G) \setminus V(T)$
- $f(uv)$ est minimum

Ajouter uv à T

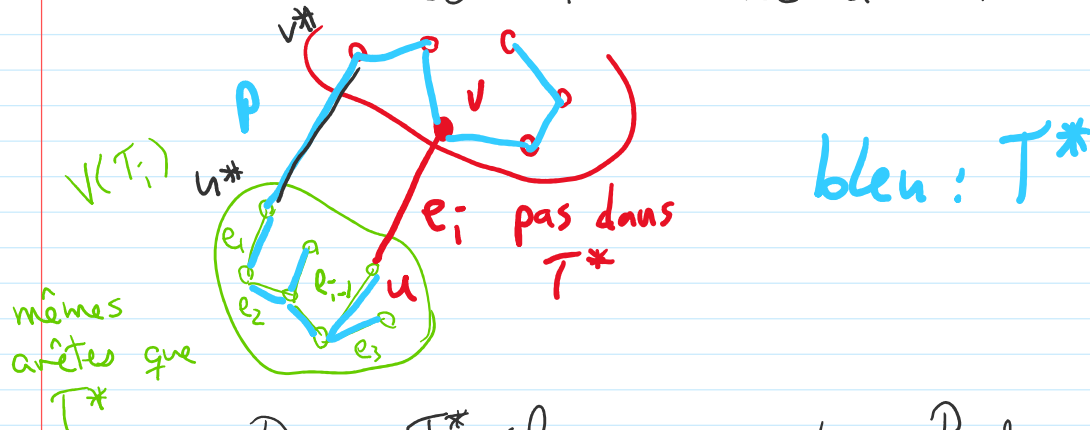


Thm: l'algorithme de Prim trouve toujours un ACM
(si G est connexe)

Preuve: soit T l'arbre retourné par Prim et soient
 $e_1, e_2, \dots, e_{n-1}$ les arêtes ajoutées, dans l'ordre.
Soit T^* un ACM.

Soit i l'indice minimum tel que $e_i \notin E(T^*)$.
Si i n'existe pas, alors $T = T^*$ et la preuve est finie.

Soit T_i l'arbre de Prim avant l'ajout de e_i .
On dénote $e_i = uv$ avec $u \in V(T_i)$ et $v \notin V(T_i)$.



Dans T^* , il y a un chemin P de u à v , et ce chemin ne passe pas par uv .

Soit u^*v^* une arête de P t.q. $u^* \in V(T_i)$, $v^* \notin V(T_i)$.

Par le choix glouton de Prim, on a

$f(uv) \leq f(u^*v^*)$, car u^*v^* était une arête candidate au moment d'ajouter uv .

Soit $\hat{T}$ l'arbre obtenu de T^* en retirant u^*v^* et en ajoutant uv .

Le coût de $\hat{T}$ est $\leq$ au coût de T^* .

De plus, $\hat{T}$ a $n-1$ arêtes et est connexe

(pour le voir, notez que $P+uv$ forme un cycle et que retirer une arête d'un cycle ne déconnecte pas un graphe).

Donc, $\hat{T}$ est aussi un ACM.

De plus, $\hat{T}$ a une arête de plus en commun avec notre T .

En répétant cet argument au plus $n-1$ fois, on obtient un ACM dont toutes les arêtes sont en commun avec T . Donc T est optimal.